PROJECT PROPOSAL

SmartStock: A Smart Inventory and Sales Management System for Small Businesses

Technology Stack: Java Spring Boot, React.js, MySQL

Project Type: Full-Stack Web Application

Team Member: Kasun Nadeera & Rihaz Ramzaan

Program: Mentorship Program

Proposed Duration: 8 Weeks

---

1. Project Title

SmartStock: A Smart Inventory and Sales Management System for Small Businesses

2. Introduction and Background

Small businesses such as retail shops, grocery stores, stationery shops, and electronic stores need to manage their products, stock levels, suppliers, purchases, and sales efficiently. Many small businesses still rely on manual records, spreadsheets, or disconnected systems to manage these operations.

These methods can lead to inaccurate stock records, calculation errors, difficulties in tracking sales, delayed identification of low-stock products, and time-consuming report preparation.

The proposed project, SmartStock, is a web-based Inventory and Sales Management System designed to help small businesses manage their daily operations through a centralized digital platform.

The system will integrate inventory management, purchasing, sales processing, supplier management, reporting, and business analytics. Advanced features such as role-based access control, automated stock alerts, sales analytics, audit logs, and predictive stock insights will improve operational efficiency and support better business decisions.

3. Problem Statement

Small businesses frequently experience the following problems:

- Difficulty maintaining accurate and up-to-date inventory records.
- Errors in manually calculating sales totals, discounts, taxes, and profits.
- Lack of real-time visibility into available stock.
- Delays in identifying products that need to be reordered.
- Difficulty tracking supplier information and purchase history.
- Time-consuming preparation of sales and inventory reports.
- Limited visibility into business performance and product sales trends.
- Inadequate access control and accountability when multiple employees use the same system.

Therefore, a centralized and user-friendly system is needed to automate these processes, minimize errors, improve inventory accuracy, and provide useful business insights.

4. Proposed Solution

SmartStock will provide a centralized web application through which authorized users can manage inventory, suppliers, purchases, sales, and reports.
The system will automatically update stock quantities when purchases, sales, and returns are processed. It will generate alerts for low-stock products and provide dashboards containing important business statistics.
The application will also include secure authentication, role-based permissions, data validation, audit logging, and advanced reporting capabilities.
The system will be developed using React.js for the frontend, Java Spring Boot for the backend REST API, and MySQL for database management.

5. Project Objectives

The main objectives are to:

1. Develop a centralized platform for managing products and inventory.
2. Automate sales transactions and stock quantity updates.
3. Manage suppliers, purchase orders, and goods received.

4. Generate accurate sales invoices and transaction records.
5. Identify low-stock products and support timely reordering.
6. Generate sales, inventory, and profitability reports.
7. Provide business analytics through an interactive dashboard.
8. Implement secure authentication and role-based access control.
9. Maintain audit records of important user activities.
10. Apply software engineering best practices, automated testing, and secure API development.
11. Develop a responsive application that can be accessed from desktop and mobile browsers.
12. Deliver a functional, tested, and documented application within the program period.

6. Target Users

The system will support the following user roles.

6.1 System Administrator

- Manage user accounts and roles.
- Configure business settings and system preferences.
- View audit logs and system-wide reports.
- Manage permissions and access restrictions.

6.2 Business Owner / Manager

- View business performance dashboards.
- Manage products, suppliers, and purchasing.
- Monitor stock levels and sales.
- View profit summaries and reports.
- Approve selected transactions where required.

6.3 Sales Staff

- Search and view available products.
- Process customer sales.
- Generate invoices and receipts.
- Process permitted sales returns.

6.4 Inventory Staff

- Update product and stock records through authorized workflows.
- Record received goods.
- Monitor stock levels and expiry information where applicable.
- Review inventory movements and reorder alerts.

7. Functional Requirements and Main Modules

7.1 User Authentication and Authorization

- Secure login and logout.
- User registration or administrator-created accounts.
- Password hashing and secure password storage.
- Role-based access control (RBAC).
- Protected frontend routes and backend endpoints.
- Access and refresh token management.
- Password reset workflow.
- Account activation, deactivation, and session security controls.

7.2 Product and Category Management

- Create, view, update, and deactivate products.
- Organize products into categories and subcategories.

- Store product name, SKU, barcode, description, selling price, cost price, and reorder level.
- Search and filter products by name, category, SKU, or barcode.
- Prevent duplicate product identifiers.
- Support product images.
- Support barcode-based product lookup.
- Import product records from CSV files and export product lists.

7.3 Inventory Management

- Display current stock quantities.
- Automatically update stock after purchases and sales.
- Record stock adjustments with reasons.
- Maintain a stock movement history.
- Generate low-stock and out-of-stock alerts.
- Support stock transfers between locations as an advanced extension.
- Support batch and expiry-date tracking for suitable product types.
- Maintain a consistent inventory ledger.
- Prevent unauthorized or invalid stock adjustments.

7.4 Supplier Management

- Register and maintain supplier details.
- Store supplier contact information and addresses.
- View supplier-specific purchase histories.
- Track outstanding purchase orders.
- Search and filter suppliers.
- Deactivate suppliers without deleting historical transactions.

7.5 Purchase and Goods Receiving Management

- Create and manage purchase orders.
- Add products, quantities, and purchase costs.
- Calculate order totals automatically.
- Track order statuses, such as Draft, Submitted, Partially Received, Received, and Cancelled.
- Record goods received from suppliers.
- Update inventory only when goods are received.
- Support partial deliveries.
- Maintain purchase history.
- Generate purchase order documents.

7.6 Sales and Invoice Management

- Create sales transactions through a simple point-of-sale interface.
- Search products by name, SKU, or barcode.
- Add products to a shopping cart.
- Calculate subtotals, discounts, applicable taxes, and final totals.
- Support cash and other configured payment methods.
- Generate unique invoice numbers.
- Update stock automatically after successful sales.
- Print or download invoices and receipts as PDF.
- Maintain customer and sales history.
- Support full or partial returns according to configured business rules.
- Record refunds and restore stock when appropriate.

7.7 Customer Management

- Register and manage customer information.
- Store customer contact details.
- View customer purchase history.
- Search customers by name or contact number.
- Associate sales invoices with customers.
- Support optional customer loyalty points as an extension.

7.8 Dashboard and Business Analytics

The dashboard will display key performance indicators, including:

- Total sales for the selected period.
- Number of completed sales.
- Estimated gross profit.
- Inventory valuation.
- Low-stock and out-of-stock product counts.
- Best-selling products.
- Sales trends over time.
- Category-wise sales performance.
- Recent transactions.
- Outstanding purchase orders.

Interactive charts and date filters will help business owners identify trends and make informed decisions.

7.9 Reports and Exporting

The system will provide:

- Daily, weekly, and monthly sales reports.
- Product-wise sales reports.

- Inventory stock reports.
- Low-stock reports.
- Purchase history reports.
- Supplier transaction reports.
- Sales return reports.
- Gross profit summaries.
- Stock movement reports.
- Inventory valuation reports.

Users with appropriate permissions will be able to filter reports by date, product, category, or supplier and export supported reports to PDF and CSV/Excel-compatible formats.

7.10 Notifications and Alerts

- Low-stock notifications.
- Out-of-stock alerts.
- Purchase order status notifications.
- Product expiry alerts where expiry tracking is enabled.
- In-app notifications for important business events.
- Optional email notifications through an email service.

7.11 Audit Logs and Transaction History

- Record important user actions.
- Track stock adjustments and transaction changes.
- Record the user, action, timestamp, and relevant entity.
- Maintain a history of sales, purchases, and returns.
- Restrict audit-log access to authorized users.
- Preserve transaction history rather than silently deleting completed transactions.

7.12 Business Settings

- Configure business name, address, and contact details.
- Configure currency, tax rates, invoice numbering, and receipt details.
- Configure reorder levels and notification preferences.
- Configure user permissions and selected operational settings.

8. Advanced Features

The following features will make SmartStock more than a basic CRUD application.

8.1 Intelligent Reorder Recommendations

Analyze sales velocity, current stock, supplier lead times, and reorder levels to suggest which products should be reordered and in what quantities.

8.2 Sales Forecasting

Use historical sales data to estimate future product demand. A lightweight forecasting method can be implemented initially, with more advanced statistical methods added if sufficient historical data is available.

8.3 Barcode-Based Operations

Enable barcode scanning through a compatible scanner or device camera where supported, allowing staff to identify products quickly during sales and stock receiving.

8.4 Multi-Branch Inventory

Extend the system to manage multiple branches or warehouses, including stock transfers and location-specific inventory records.

8.5 Advanced Inventory Tracking

Support batch numbers, expiry dates, and configurable stock valuation methods, such as FIFO, where appropriate for the selected business rules.

8.6 Concurrency and Transaction Safety

Use database transactions and appropriate locking or concurrency controls to prevent overselling, duplicate processing, and inconsistent inventory when multiple users perform transactions simultaneously.

8.7 Secure API Design

Implement input validation, consistent error responses, authorization checks, secure configuration, rate limiting where appropriate, and protection against common web vulnerabilities.

8.8 Data Import, Export, and Backup

Provide controlled CSV imports, report exports, and a documented database backup and restoration procedure.

8.9 API Documentation

Use OpenAPI/Swagger to document REST endpoints, request formats, responses, and authentication requirements.

8.10 Optional AI-Powered Insights

As an extension, use an AI service to summarize sales trends or explain stock recommendations. This feature will be optional and will not be required for the core system to function.

9. Proposed Technology Stack

Frontend

- React.js
- JavaScript or TypeScript
- HTML5 and CSS3
- Bootstrap or Material UI
- Axios for API communication
- React Router for navigation
- Recharts for data visualization

Backend

- Java
- Spring Boot
- Spring Web for REST API development

- Spring Data JPA and Hibernate
- Spring Security
- JWT-based authentication
- Bean Validation
- Maven for dependency management
- OpenAPI/Swagger for API documentation

Database

- MySQL
- Relational database design
- Primary and foreign key constraints
- Indexing for commonly searched fields
- Database transactions and appropriate constraints

Development and Deployment Tools

- Visual Studio Code or IntelliJ IDEA
- Git and GitHub
- Postman for API testing
- JUnit and Mockito for backend testing
- React Testing Library for frontend testing
- Docker and Docker Compose
- GitHub Actions for optional CI/CD
- A suitable cloud platform or local server for deployment

10. Proposed System Architecture

The application will follow a layered architecture.

Presentation Layer: React.js provides the user interface, forms, dashboards, reports, and responsive layouts.

API Layer: Spring Boot REST controllers receive HTTP requests and return structured responses.

Business Logic Layer: Services handle product rules, purchasing, sales, invoice generation, stock updates, permissions, and reporting.

Data Access Layer: Spring Data JPA repositories communicate with MySQL.

Database Layer: MySQL stores users, products, suppliers, customers, purchases, sales, invoices, inventory movements, and audit records.

External Services: Optional integrations will provide email notifications, barcode support, and AI-based insights.

Typical request flow:

React Frontend → Spring Boot REST API → Service Layer → Repository Layer → MySQL Database

The backend will remain responsible for critical business rules and authorization. The frontend will not be trusted to calculate or authorize financial and inventory transactions independently.

11. Database Design

The proposed database will contain the following main entities:

1. Users
2. Roles
3. Permissions
4. User Roles / Role Permissions
5. Products
6. Categories
7. Suppliers
8. Customers
9. Warehouses or Branches
10. Purchase Orders

11. Purchase Order Items
12. Goods Receipts
13. Goods Receipt Items
14. Sales
15. Sale Items
16. Payments
17. Sales Returns
18. Sales Return Items
19. Inventory Balances
20. Stock Movements
21. Product Batches
22. Notifications
23. Audit Logs
24. Business Settings

Relationships will be defined using primary keys, foreign keys, and suitable constraints. For example, each sale can contain multiple sale items, each purchase order can contain multiple products, and each stock movement will reference the relevant product and transaction.

The final database schema will be refined during design to prevent unnecessary duplication and preserve transaction history.

12. Security and Quality Requirements

The application will incorporate the following non-functional requirements:

- **Security:** Secure password hashing, authorization, server-side validation, and protected API endpoints.
- **Performance:** Pagination, database indexing, efficient queries, and appropriate loading states.

- **Reliability:** Transaction-safe stock updates and consistent sales and purchasing records.
- **Usability:** Clear navigation, meaningful error messages, and an easy-to-use sales interface.
- **Maintainability:** Layered architecture, reusable React components, and consistent coding conventions.
- **Scalability:** Modular backend design that can support additional modules and branches.
- **Compatibility:** Support for modern desktop and mobile browsers.
- **Testability:** Automated tests for important business rules and transaction workflows.
- **Data Integrity:** Constraints and validation to prevent invalid prices, quantities, and duplicate transactions.
- **Recoverability:** Documented backup and restoration procedures.

13. Project Methodology

The project will follow an iterative Agile development approach.

Development will be divided into manageable stages, with each stage producing a testable component. GitHub will be used for version control, feature branches, pull requests, and code reviews.

Regular discussions between team members will help identify issues early and ensure that the project remains within the available timeframe.

14. Proposed Timeline

The following schedule assumes an eight-week development period.

Week 1 – Planning and Requirements

- Finalize project scope and objectives.
- Identify users and use cases.
- Prepare wireframes and system architecture.

- Design the initial database schema.
- Create the GitHub repository and development environment.

Week 2 – Backend Foundation and Database

- Set up the Spring Boot application.
- Configure MySQL and JPA.
- Implement product, category, and user entities.
- Create initial REST APIs.
- Add validation and error-handling conventions.

Week 3 – Authentication and Product Management

- Implement authentication and authorization.
- Configure roles and permissions.
- Develop product and category management.
- Implement search, filtering, and pagination.

Week 4 – Suppliers, Purchasing, and Inventory

- Implement supplier management.
- Develop purchase order workflows.
- Record goods received.
- Implement stock movements and low-stock alerts.

Week 5 – Sales and Invoicing

- Develop the point-of-sale interface.
- Implement sales transactions and invoice numbering.
- Add payment recording and stock updates.
- Generate invoice PDFs.
- Test transaction consistency and overselling prevention.

Week 6 – Dashboard, Reports, and Advanced Features

- Develop dashboard charts and KPIs.

- Implement core reports and exports.
- Add audit logs and in-app notifications.
- Begin implementing the highest-priority advanced feature.

Week 7 – Integration, Testing, and Refinement

- Integrate frontend and backend modules.
- Conduct functional, security, and usability testing.
- Fix bugs and improve performance.
- Complete API documentation.

Week 8 – Deployment and Final Presentation

- Configure Docker-based deployment if feasible.
- Prepare database backup and setup instructions.
- Complete user documentation.
- Deploy the application.
- Prepare the final demonstration and project report.

Advanced extensions such as multi-branch transfers, forecasting, and AI-generated insights will be completed according to remaining time and priority.

15. Team Responsibilities

The two team members will collaborate on requirements, design, integration, testing, and documentation while maintaining clear individual responsibilities.

Team Member 1 – Backend and Database Focus

- Develop Spring Boot REST APIs.
- Implement business logic and validation.
- Design and maintain the MySQL database.
- Implement authentication, authorization, and transaction handling.
- Write backend tests and API documentation.

Team Member 2 – Frontend and User Experience Focus

- Develop the React application.
- Implement responsive layouts and navigation.
- Build forms, tables, dashboards, and sales screens.
- Integrate frontend components with backend APIs.
- Write frontend tests and improve usability.

Shared Responsibilities

- Review code through GitHub pull requests.
- Collaborate on system architecture and database decisions.
- Test end-to-end workflows.
- Resolve integration issues.
- Prepare documentation and the final presentation.

Both members will gain experience with frontend and backend technologies through code reviews, pair programming, and shared testing.

16. Expected Outcomes

At the end of the project, the expected deliverables are:

- A functional full-stack inventory and sales management web application.
- Secure user authentication and role-based permissions.
- Product, category, supplier, and customer management.
- Purchasing and inventory tracking workflows.
- Sales processing and invoice generation.
- Dashboard analytics and business reports.
- Automated stock alerts and transaction history.
- Audit logging and data integrity controls.
- Documented REST APIs.
- Automated tests for critical operations.

- Source code maintained in a GitHub repository.
- Deployment and setup documentation.
- A final project report and demonstration.

The application should demonstrate how software engineering can reduce manual work, improve stock accuracy, and help small businesses make better operational decisions.

17. Project Scope and Limitations

The initial release will focus on a single small business with one primary inventory location. The architecture will allow future expansion to multiple branches.

Advanced features will be implemented progressively according to available time. Sales forecasting will require sufficient historical data to produce meaningful results. AI integrations, online payment gateways, accounting-system integrations, and complex multi-branch workflows may remain optional extensions.

The application will not claim to replace a complete accounting or enterprise resource planning system. Financial reports will be limited to the information captured by the implemented sales, purchase, and inventory workflows.

18. Conclusion

SmartStock aims to provide a practical, secure, and user-friendly inventory and sales management solution for small businesses. By combining Java Spring Boot, React.js, and MySQL, the project will demonstrate full-stack development, database design, REST API integration, authentication, transaction management, testing, and deployment.

The inclusion of advanced capabilities such as automated stock alerts, audit logging, business analytics, and intelligent reorder recommendations will provide opportunities to apply modern software engineering practices while maintaining a realistic core scope.

The proposed project is suitable for collaborative development by two team members and can be delivered in stages within the program period, subject to the final scope agreed upon with the instructor.

END